

2dosumi Raspberry Pi デプロイ手順

Windows から Raspberry Pi へ同期し、校正済み設定を守りながら Web UI を systemd と Tailscale Serve で運用するための実行手順です。

このPDFで扱う範囲

初回デプロイ、更新デプロイ、Tailscale公開、センサー確認、校正、systemdの運用確認。OSイメージ作成やTailscaleアカウント登録は前提外です。

前提

- ☐ Windows から ssh/scp で Pi に接続できる
- ☐ Pi は systemd が使える Raspberry Pi OS 系
- ☐ Tailscale は Pi 側でログイン済み、または後で設定する
- ☐ GPIO 配線は DATA=D6、SCK=D5、ブザー=D13 前提

既定値

項目	値
HostName	192.168.137.105
User	yuki
RemoteDir	/home/yuki/2dosumi
Port	8080

全体の流れ

順番	作業	結果
1	Windowsからdeploy_pi.ps1を実行	コードだけをPiへ同期
2	Pi上で.venv作成と依存更新	Python環境をアプリ専用に固定
3	systemdサービスを再起動	起動時にWeb UIが自動起動
4	Tailscale Serveを有効化	ブラウザから安全にアクセス
5	センサー確認と校正	検知に使う重量値を本番化

1. 初回デプロイ

まず Windows 側のプロジェクトフォルダから Pi へ接続できることを確認します。

```
ssh yuki@192.168.137.105
```

接続できたら、Windows 側でデプロイスクリプトを実行します。

```
.\scripts\deploy_pi.ps1 -HostName 192.168.137.105 -User yuki -RemoteDir /home/yuki/2dosumi
```

重要: Pi側の設定を守る

config/pi.aggregate.json は Pi に存在しない初回だけ配置します。既にある場合は上書きしません。config/secrets.json も通常のデプロイ対象外です。

スクリプトが行うこと

- ☐ twodosumi、scripts、docs、requirements-pi.txt、README.md を同期
- ☐ Pi に config/pi.aggregate.json がない場合だけ初期設定を配置
- ☐ .venv を作成または更新し、requirements-pi.txt をインストール
- ☐ twodosumi-web.service をインストールまたは再起動
- ☐ http://127.0.0.1:8080/healthz を確認

作業ツリーに未コミット変更がある状態で実機テストしたい場合だけ、明示的に許可します。

```
.\scripts\deploy_pi.ps1 -AllowDirty
```

venv作成で失敗した場合

Pi側で python3-venv が入っていない可能性があります。次を実行してから、Windows側でデプロイを再実行します。

```
ssh yuki@192.168.137.105
```

```
sudo apt-get update
```

```
sudo apt-get install -y python3-venv
```

2. Web UIをTailscaleで公開

Flask は Pi の localhost だけで待ち受けます。外からのアクセスは Tailscale Serve を使います。

```
tailscale serve --bg 8080
tailscale serve status
```

初回トークン

初回起動時に config/secrets.json が作成され、web_ui_token が発行されます。Web UIでこのトークンを入力します。

```
journalctl -u twodosumi-web.service --no-pager -n 50
```

3. センサー配線の前提

HX711	Raspberry Pi
DT / DOUT	GPIO6 / board.D6
SCK / CLK	GPIO5 / board.D5
VCC	3.3V
GND	GND

電圧注意

まず HX711 VCC は 3.3V で使います。Raspberry Pi の GPIO に 5V 信号を入れないでください。

4. 最初のセンサー確認

```
cd /home/yuki/2dosumi
. .venv/bin/activate
python -m twodosumi check-sensor --config config/pi.aggregate.json --samples 10
```

成功時は ok=True、samples=10/10、raw の min/max/median が表示されます。Web UI の センサー確認 ボタンでも同じ確認ができます。

よくある失敗

HX711 DOUT stayed HIGH が出る場合は、HX711の電源、GND、DT/DOUT、SCK/CLK、ロードセルブリッジ配線を確認します。

5. 校正

検知精度の中心になる作業です。ベッドを空にしてゼロ校正し、既知の重さを載せてスケール校正します。

ゼロ校正

```
python -m twodosumi calibrate-zero --config config/pi.aggregate.json
```

重量校正

```
python -m twodosumi calibrate-scale --config config/pi.aggregate.json --known-kg 8
```

known-kg

例の 8 は実際に載せた既知重量 kg に置き換えます。校正後は再度センサー確認し、weight_median_kg が現在の荷重に近いことを確認します。

6. サービス運用確認

```
systemctl status twodosumi-web.service --no-pager  
journalctl -u twodosumi-web.service -f  
tailscale serve status  
curl http://127.0.0.1:8080/healthz
```

サービスを手動で入れ直す場合

```
cd /home/yuki/2dosumi  
APP_DIR=/home/yuki/2dosumi APP_USER=yuki PORT=8080 ./scripts/install_twodosumi_web_service.sh
```

Web UIを手動起動してデバッグする場合

```
cd /home/yuki/2dosumi  
.  
. .venv/bin/activate  
python -m twodosumi web --config config/pi.aggregate.json --secrets config/secrets.json --host 127.0.0.1 --port 8080
```

7. 日常運用と更新

Web UI へ Tailscale 経由でアクセスし、web_ui_token を入力して操作します。

- ☐ センサー確認 を実行
- ☐ ライブ重量とステータスを確認
- ☐ 予定アラームを設定
- ☐ 開始 を押して検知を開始

コード更新時

```
.\scripts\deploy_pi.ps1
```

未コミットの作業中変更を実機に送る場合だけ次を使います。

```
.\scripts\deploy_pi.ps1 -AllowDirty
```

トラブルシューティング早見表

症状	見る場所	対応
Web UIが開かない	systemctl status / curl / tailscale serve status	サービス起動、ポート8080、Tailscale Serveを確認
認証できない	journalctl / config/secrets.json	web_ui_token を確認して再入力
センサー値が出ない	check-sensor のエラー	HX711電源、GND、DOUT、SCK、ブリッジ配線を確認
重量がずれる	weight_median_kg	ゼロ校正と重量校正をやり直す
更新で設定が戻る	Pi側 config	通常デプロイでは config を上書きしない。手動コピーを避ける

運用ルール

Pi 上の config/pi.aggregate.json と config/secrets.json は本番状態です。校正値、Web UI設定、Webhook URL、トークンを守るため、通常の更新ではPC側から上書きしません。